SOC Detection & Monitoring Lab Implementation Plan v1.0

Project: SOC Detection & Monitoring Lab
Document: Implementation Plan / Build Runbook
Version: v1.0
Baseline Date: 2026-08-24
Input Baseline: Requirements v1.0 → HLD v1.0 → LLD v1.0
Initial Status: `READY TO START PHASE 0`

---

1. Implementation 목표

본 구축의 최종 목표는 다음 End-to-End Pipeline을 **실제 동작 결과와 Evidence로 증명**하는 것이다.

```
Attacker
10.77.20.20
      ↓
Gateway
10.77.20.1 / 10.77.30.1
      ↓
Victim
10.77.30.20
      ↓
Hyper-V Port Mirroring
      ↓
Sensor Monitoring NIC
NO IP
      ↓
AF_PACKET
      ↓
Suricata 8.0.6
      ↓
Detection Rule
      ↓
eve.json
      ↓
Wazuh Agent
      ↓
Wazuh Manager
      ↓
Wazuh Indexer
      ↓
Wazuh Dashboard
      ↓
SOC Analyst
      ↓
Raw Event + PCAP + Rule
      ↓
MITRE ATT&CK
      ↓
Verdict
      ↓
Detection Tuning
      ↓
Re-test
```

```
      ↓
Evidence
```

---

2. 구현 절대 원칙

구축 순서는 다음을 따른다.

```
Network
   ↓
Routing
   ↓
Firewall
   ↓
Traffic
   ↓
Port Mirroring
   ↓
tcpdump
   ↓
Suricata
   ↓
Detection
   ↓
EVE
   ↓
Wazuh
   ↓
SOC Analysis
```

Critical Rule

> **Packet Visibility가 검증되기 전에는 Suricata 구축을 완료한 것으로 간주하지 않는다.**

또한:

> **Suricata Alert가 로컬에서 확인되기 전에는 Wazuh Integration으로 넘어가지 않는다.**

---

3. Version Baseline

Component	Baseline
Suricata	**8.0.6**
Snort	**3.12.2.0**
libDAQ	**3.0.27**
Wazuh	**4.14.7**
ATT&CK	**19.2**
Ubuntu Sensor	22.04.x
Ubuntu Gateway	22.04.x
Ubuntu Victim	22.04.x

Component	Baseline
Kali	2026.2
Docker Desktop	LLD Baseline / Runtime 확인
WSL	≥ 2.1.5

Suricata 8.0.6 문서는 현재 공식 8.0.6 User Guide를 제공하고 있으며 AF_PACKET, EVE, PCAP 분석 등을 지원한다. Snort 공식 다운로드에는 Snort 3.12.2.0과 libDAQ 3.0.27이 제공된다. Wazuh 공식 Docker 가이드는 `v4.14.7` single-node 배포를 제시한다.

Version Drift Policy

실행 시 다른 최신 버전이 조회되더라도 자동 Upgrade하지 않는다.

```
VERSION-DRIFT-xxx

Baseline:
Current:
Impact:
Decision:
KEEP_BASELINE / UPGRADE
ADR:
YES / NO
```

---

4. Phase Master Table

Phase	목적	Gate
0	Host 준비상태 확인	GATE-HOST-01
1	Repository Baseline	GATE-REPO-01
2	Hyper-V vSwitch	GATE-NET-INFRA-01
3	VM Provisioning	GATE-VM-01
4	IP / Interface Mapping	GATE-IP-01
5	Routing	GATE-ROUTE-01
6	nftables	GATE-FW-01
7	Port Mirroring	GATE-MIRROR-CONFIG-01
8	Packet Visibility	**GATE-NET-01**
9	Suricata 설치	GATE-SURI-INSTALL-01
10	Suricata 설정	GATE-SURI-01
11	Detection	GATE-DETECT-01
12	PCAP	GATE-PCAP-01
13	Snort 설치	GATE-SNORT-INSTALL-01
14	Snort Validation	GATE-SNORT-01
15	Wazuh Host 준비	GATE-WAZUH-HOST-01
16	Wazuh Docker	GATE-WAZUH-01

Phase	목적	Gate
17	Sensor Agent	GATE-AGENT-SENSOR-01
18	Victim Agent	GATE-AGENT-VICTIM-01
19	Suricata→Wazuh	GATE-SIEM-01
20	Dashboard	GATE-DASHBOARD-01
21	Attack Scenario	GATE-ATTACK-01
22	SOC Investigation	GATE-ANALYSIS-01
23	ATT&CK	GATE-ATTACK-MAP-01
24	FP 분석	GATE-FP-01
25	Rule Tuning	GATE-TUNE-CONFIG-01
26	Re-test	GATE-TUNE-01
27	End-to-End	**GATE-E2E-01**
28	Evidence	GATE-EVIDENCE-01
29	GitHub Portfolio	GATE-PORTFOLIO-01
30	Final Release	GATE-RELEASE-01

5. Phase 0 — Host Readiness

Objective

Windows Host가 Hyper-V + WSL2 + Docker 기반 SOC Lab을 운영할 수 있는 상태인지 검증한다.

Prerequisite

없음.

Tasks

- Windows Version 확인
- Hyper-V 확인
- Hardware Virtualization 확인
- WSL 확인
- Docker 확인
- Git 확인
- RAM / Storage 확인
- PowerShell 관리자 권한 확인

Commands

[PowerShell - Administrator]

```
Get-ComputerInfo | Select-Object WindowsProductName, WindowsVersion, Os
BuildNumber
systeminfo
```

Hyper-V

```
Get-WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V-All
Get-VMHost
```

Virtualization

```
Get-CimInstance Win32_Processor |
Select-Object Name, VirtualizationFirmwareEnabled, SecondLevelAddressTr
anslationExtensions
```

WSL

```
wsl --version
wsl -l -v
```

Docker Desktop WSL2 backend은 최소 WSL 2.1.5를 요구하며 최신 WSL 사용을 권고한다.

Docker

```
docker version
docker compose version
docker info
```

Git

```
git --version
```

RAM

```
Get-CimInstance Win32_ComputerSystem |
Select-Object TotalPhysicalMemory
```

Disk

```
Get-Volume |
Select-Object DriveLetter, FileSystemLabel,
```

```
@{N='FreeGB';E={[math]::Round($_.SizeRemaining/1GB,2)}},
@{N='TotalGB';E={[math]::Round($_.Size/1GB,2)}}
```

Runtime Gates

RG-001

```
Host Windows Build
```

RG-002

```
WSL Exact Version
```

RG-003

```
VirtualizationFirmwareEnabled = True
```

RG-004

권장:

```
Host RAM >= 32 GB
```

RG-005

권장:

```
SOC Lab Free Storage >= 200 GB
```

Evidence

```
EV-HOST-001
```

포함:

```
Windows Version
Hyper-V Status
WSL Version
Docker Version
RAM
Disk
```

Completion Gate

GATE-HOST-01

PASS 조건

```
Hyper-V            PASS
Virtualization     PASS
WSL2               PASS
Docker             PASS
Git                PASS
Storage            PASS
```

RAM이 32GB 미만이어도 구축 자체는 가능하지만, 동시 실행 대신 Phase별 Service Start 전략을 사용한다.

---

6. Phase 1 — Repository Baseline

Objective

향후 Configuration, Rule, Test, Evidence, Incident Report가 하나의 Repository에서 추적되도록 한다.

Input

```
C:\SOC-Lab\repo\soc-detection-monitoring-lab
```

Commands

[PowerShell]

```
New-Item -ItemType Directory -Force -Path "C:\SOC-Lab\repo\soc-detectio
n-monitoring-lab"
Set-Location "C:\SOC-Lab\repo\soc-detection-monitoring-lab"

$dirs = @(
"docs\01-requirements",
"docs\02-architecture",
"docs\03-design",
"docs\04-deployment",
"docs\05-testing",
"docs\06-detection\comparisons",
"docs\07-investigation",
"docs\08-incident",
"docs\09-troubleshooting",
"infrastructure\hyper-v",
```

```
"infrastructure\network",
"infrastructure\docker",
"suricata\config",
"suricata\rules",
"suricata\scripts",
"snort\config",
"snort\rules",
"snort\scripts",
"wazuh\config",
"wazuh\docker",
"attack-lab\scenarios",
"attack-lab\scripts",
"pcaps\samples",
"pcaps\metadata",
"tests",
"evidence",
"scripts"
)

$dirs | ForEach-Object {
    New-Item -ItemType Directory -Force -Path $_ | Out-Null
}

New-Item README.md -ItemType File -Force
New-Item .gitignore -ItemType File -Force
New-Item .env.example -ItemType File -Force
```

.gitignore

```
@'
.env
*.key
*.pem
*.p12
*.pfx
secrets/
credentials/
runtime/
logs/
pcaps/raw/
pcaps/archive/
*.pcap
*.pcapng
'@ | Set-Content .gitignore
```

Git

```
git init
git add .
git commit -m "chore: initialize SOC lab repository"
```

Evidence

```
EV-REPO-001
```

Rollback

Git 초기화만 취소할 경우:

```
[DESTRUCTIVE]
Remove-Item -Recurse -Force ".git"
```

Completion Gate

GATE-REPO-01

```
Repository structure   PASS
.gitignore             PASS
.env ignored           PASS
Baseline commit        PASS
```

---

7. Phase 2 — Hyper-V Virtual Network

Objective

Attack / Victim / Management 영역을 물리적으로 분리된 Hyper-V vSwitch로 생성한다.

Hyper-V의 `New-VMSwitch`는 Internal/Private 등 Switch Type을 지원한다.

Commands

[PowerShell - Administrator]

```
New-VMSwitch -Name "soc-vsw-mgmt" -SwitchType Internal
New-VMSwitch -Name "soc-vsw-attack" -SwitchType Private
New-VMSwitch -Name "soc-vsw-victim" -SwitchType Private
```

확인:

```
Get-VMSwitch |
Where-Object Name -like "soc-vsw-*" |
Format-Table Name, SwitchType
```

Windows MGMT IP

Internal Switch에 생성되는 Host Adapter:

```
vEthernet (soc-vsw-mgmt)
```

설정:

```
Set-NetIPInterface `
  -InterfaceAlias "vEthernet (soc-vsw-mgmt)" `
  -Dhcp Disabled

New-NetIPAddress `
  -InterfaceAlias "vEthernet (soc-vsw-mgmt)" `
  -IPAddress 10.77.10.10 `
  -PrefixLength 24
```

검증:

```
Get-NetIPAddress `
  -InterfaceAlias "vEthernet (soc-vsw-mgmt)" `
  -AddressFamily IPv4
```

Evidence

```
EV-NET-INFRA-001
```

Rollback

```
[DESTRUCTIVE]
Remove-VMSwitch -Name "soc-vsw-victim" -Force
Remove-VMSwitch -Name "soc-vsw-attack" -Force
Remove-VMSwitch -Name "soc-vsw-mgmt" -Force
```

Completion Gate

GATE-NET-INFRA-01

```
soc-vsw-mgmt    Internal   PASS
soc-vsw-attack  Private    PASS
soc-vsw-victim  Private    PASS
Host 10.77.10.10           PASS
```

---

8. Phase 3 — VM Provisioning

Objective

LLD에서 정의한 4개 VM을 생성한다.

Microsoft `New-VM`은 Generation, Memory, VHD 경로·크기 등을 지정할 수 있고 `Add-VMNetworkAdapter`로 추가 Adapter를 연결할 수 있다.

ISO Baseline

```
C:\SOC-Lab\iso\ubuntu-22.04.5-live-server-amd64.iso
C:\SOC-Lab\iso\kali-linux-2026.2-installer-amd64.iso
```

ISO 존재 여부:

```
Test-Path "C:\SOC-Lab\iso\ubuntu-22.04.5-live-server-amd64.iso"
Test-Path "C:\SOC-Lab\iso\kali-linux-2026.2-installer-amd64.iso"
```

---

soc-gateway

[PowerShell - Administrator]

```
New-VM `
 -Name "soc-gateway" `
 -Generation 2 `
 -MemoryStartupBytes 1GB `
 -NewVHDPath "C:\SOC-Lab\vm\soc-gateway\soc-gateway.vhdx" `
 -NewVHDSizeBytes 16GB

Set-VMProcessor -VMName "soc-gateway" -Count 1
Set-VMMemory -VMName "soc-gateway" -DynamicMemoryEnabled $false

Rename-VMNetworkAdapter `
 -VMName "soc-gateway" `
 -Name "Network Adapter" `
```

```
 -NewName "nic-mgmt"

Connect-VMNetworkAdapter `
 -VMName "soc-gateway" `
 -Name "nic-mgmt" `
 -SwitchName "soc-vsw-mgmt"

Add-VMNetworkAdapter `
 -VMName "soc-gateway" `
 -Name "nic-attack" `
 -SwitchName "soc-vsw-attack"

Add-VMNetworkAdapter `
 -VMName "soc-gateway" `
 -Name "nic-victim" `
 -SwitchName "soc-vsw-victim"

Set-VMDvdDrive `
 -VMName "soc-gateway" `
 -Path "C:\SOC-Lab\iso\ubuntu-22.04.5-live-server-amd64.iso"

Set-VMFirmware `
 -VMName "soc-gateway" `
 -EnableSecureBoot On `
 -SecureBootTemplate MicrosoftUEFICertificateAuthority
```

soc-victim

```
New-VM `
 -Name "soc-victim" `
 -Generation 2 `
 -MemoryStartupBytes 4GB `
 -NewVHDPath "C:\SOC-Lab\vm\soc-victim\soc-victim.vhdx" `
 -NewVHDSizeBytes 30GB

Set-VMProcessor -VMName "soc-victim" -Count 2
Set-VMMemory -VMName "soc-victim" -DynamicMemoryEnabled $false

Rename-VMNetworkAdapter `
 -VMName "soc-victim" `
 -Name "Network Adapter" `
 -NewName "nic-victim"

Connect-VMNetworkAdapter `
 -VMName "soc-victim" `
```

```
 -Name "nic-victim" `
 -SwitchName "soc-vsw-victim"

Set-VMDvdDrive `
 -VMName "soc-victim" `
 -Path "C:\SOC-Lab\iso\ubuntu-22.04.5-live-server-amd64.iso"

Set-VMFirmware `
 -VMName "soc-victim" `
 -EnableSecureBoot On `
 -SecureBootTemplate MicrosoftUEFICertificateAuthority
```

soc-sensor

```
New-VM `
 -Name "soc-sensor" `
 -Generation 2 `
 -MemoryStartupBytes 8GB `
 -NewVHDPath "C:\SOC-Lab\vm\soc-sensor\soc-sensor.vhdx" `
 -NewVHDSizeBytes 80GB

Set-VMProcessor -VMName "soc-sensor" -Count 4
Set-VMMemory -VMName "soc-sensor" -DynamicMemoryEnabled $false

Rename-VMNetworkAdapter `
 -VMName "soc-sensor" `
 -Name "Network Adapter" `
 -NewName "nic-mgmt"

Connect-VMNetworkAdapter `
 -VMName "soc-sensor" `
 -Name "nic-mgmt" `
 -SwitchName "soc-vsw-mgmt"

Add-VMNetworkAdapter `
 -VMName "soc-sensor" `
 -Name "nic-monitor" `
 -SwitchName "soc-vsw-victim"

Set-VMDvdDrive `
 -VMName "soc-sensor" `
 -Path "C:\SOC-Lab\iso\ubuntu-22.04.5-live-server-am64.iso"
```

주의: 실제 ISO 파일명이 `amd64`이므로 위 마지막 명령은 구현 전에 다음으로 교정하여 사용한다.

```
Set-VMDvdDrive `
 -VMName "soc-sensor" `
 -Path "C:\SOC-Lab\iso\ubuntu-22.04.5-live-server-amd64.iso"
```

이처럼 Command 검토 단계에서 발견된 Typo도 `TRB`가 아니라 문서상 교정 후 실행한다.

```
Set-VMFirmware `
 -VMName "soc-sensor" `
 -EnableSecureBoot On `
 -SecureBootTemplate MicrosoftUEFICertificateAuthority
```

---

soc-attacker

```
New-VM `
 -Name "soc-attacker" `
 -Generation 2 `
 -MemoryStartupBytes 4GB `
 -NewVHDPath "C:\SOC-Lab\vm\soc-attacker\soc-attacker.vhdx" `
 -NewVHDSizeBytes 40GB

Set-VMProcessor -VMName "soc-attacker" -Count 2
Set-VMMemory -VMName "soc-attacker" -DynamicMemoryEnabled $false

Rename-VMNetworkAdapter `
 -VMName "soc-attacker" `
 -Name "Network Adapter" `
 -NewName "nic-attack"

Connect-VMNetworkAdapter `
 -VMName "soc-attacker" `
 -Name "nic-attack" `
 -SwitchName "soc-vsw-attack"

Set-VMDvdDrive `
 -VMName "soc-attacker" `
 -Path "C:\SOC-Lab\iso\kali-linux-2026.2-installer-amd64.iso"

Set-VMFirmware `
 -VMName "soc-attacker" `
 -EnableSecureBoot Off
```

Verification

```
Get-VM |
Where-Object Name -like "soc-*" |
Format-Table Name,State,Generation,MemoryAssigned

Get-VMNetworkAdapter -VMName "soc-gateway","soc-victim","soc-sensor","s
oc-attacker" |
Format-Table VMName,Name,SwitchName,MacAddress
```

Runtime Gate

RG-005 — MAC Mapping

결과를 저장한다.

```
Get-VMNetworkAdapter `
 -VMName "soc-gateway","soc-victim","soc-sensor","soc-attacker" |
Select VMName,Name,SwitchName,MacAddress |
Export-Csv "C:\SOC-Lab\artifacts\hyperv-mac-map.csv" -NoTypeInformation
```

Completion Gate

GATE-VM-01

4개 VM 존재 + NIC 수 + vSwitch 연결이 LLD와 동일해야 한다.

---

9. Phase 4 — Network Address Configuration

Objective

VM 내부 Linux Interface와 Hyper-V NIC를 MAC으로 Mapping하고 고정 IP를 설정한다.

9.1 Interface Mapping

[soc-gateway / soc-sensor / soc-victim]

```
ip -br link
ip -br addr
ip link
```

MAC을 Phase 3의 `hyperv-mac-map.csv`와 비교한다.

Runtime 결과 기록

```
RG-003 Gateway

nic-mgmt   → <actual-interface>
nic-attack → <actual-interface>
nic-victim → <actual-interface>

RG-004 Sensor

nic-mgmt    → <actual-interface>
nic-monitor → <actual-interface>
```

9.2 Gateway Netplan

파일:

```
/etc/netplan/01-soc-lab.yaml
```

[Bash - soc-gateway]

아래 `<...>`를 Runtime Mapping 결과로 대체한다.

```yaml
network:
  version: 2
  ethernets:
    <GW_MGMT_IF>:
      addresses:
        - 10.77.10.1/24

    <GW_ATTACK_IF>:
      addresses:
        - 10.77.20.1/24

    <GW_VICTIM_IF>:
      addresses:
        - 10.77.30.1/24
```

적용:

```
sudo netplan generate
sudo netplan apply
ip -br addr
```

9.3 Victim Netplan

```yaml
network:
  version: 2
  ethernets:
    <VICTIM_IF>:
      addresses:
        - 10.77.30.20/24
      routes:
        - to: default
          via: 10.77.30.1
```

```
sudo netplan generate
sudo netplan apply
```

9.4 Sensor Netplan

MGMT에만 IP를 준다.

```yaml
network:
  version: 2
  ethernets:
    <SENSOR_MGMT_IF>:
      addresses:
        - 10.77.10.20/24

    <SENSOR_MONITOR_IF>:
      dhcp4: false
      dhcp6: false
      link-local: []
      optional: true
```

검증:

```
ip -br addr
```

Expected:

```
MGMT     10.77.10.20/24
MONITOR  NO IPv4 address
```

9.5 Kali

[Bash - soc-attacker]

```
nmcli device status
```

Runtime Interface를 `<ATTACK_IF>`로 지정.

```
sudo nmcli con add \
 type ethernet \
 ifname <ATTACK_IF> \
 con-name soc-attack \
 ipv4.method manual \
 ipv4.addresses 10.77.20.20/24 \
 ipv4.gateway 10.77.20.1 \
 ipv6.method disabled

sudo nmcli con up soc-attack
```

Completion Gate

GATE-IP-01

```
Gateway     10.77.10.1 / 10.77.20.1 / 10.77.30.1
Attacker    10.77.20.20
Victim      10.77.30.20
Sensor      10.77.10.20
Monitor     NO IP
```

---

10. Phase 5 — Gateway Routing

Objective

Attack Network와 Victim Network 사이에 명시적 L3 Routing을 활성화한다.

[Bash - soc-gateway]

```
echo 'net.ipv4.ip_forward=1' |
sudo tee /etc/sysctl.d/99-soc-lab-router.conf

sudo sysctl --system

sysctl net.ipv4.ip_forward
```

Expected:

```
net.ipv4.ip_forward = 1
```

Host Victim Return Route

[PowerShell - Administrator]

```
New-NetRoute `
 -DestinationPrefix "10.77.30.0/24" `
 -InterfaceAlias "vEthernet (soc-vsw-mgmt)" `
 -NextHop 10.77.10.1 `
 -RouteMetric 50
```

Attack Network Route는 Windows Host에 만들지 않는다.

Validation

Attacker → Gateway

```
ping -c 3 10.77.20.1
```

Victim → Gateway

```
ping -c 3 10.77.30.1
```

Gate

GATE-ROUTE-01

Gateway 양쪽 직접통신이 가능해야 한다.

---

11. Phase 6 — Gateway Firewall

Objective

Default Deny를 유지하면서 Lab Traffic만 Forward한다.

---

nftables 설치

[Bash - soc-gateway]

```
sudo apt update
sudo apt install -y nftables
sudo systemctl enable nftables
```

`/etc/nftables.conf`

실제 Interface Name으로 변경한다.

```
#!/usr/sbin/nft -f

flush ruleset

table inet soc_filter {

    chain input {
        type filter hook input priority 0;
        policy drop;

        iif "lo" accept
        ct state established,related accept

        ip saddr 10.77.10.10 ip daddr 10.77.10.1 tcp dport 22 accept
        ip protocol icmp ip saddr 10.77.10.0/24 accept
    }

    chain forward {
        type filter hook forward priority 0;
        policy drop;

        ct state established,related accept

        iifname "<GW_ATTACK_IF>" \
        oifname "<GW_VICTIM_IF>" \
        ip saddr 10.77.20.0/24 \
        ip daddr 10.77.30.20 \
        accept

        iifname "<GW_VICTIM_IF>" \
        oifname "<GW_MGMT_IF>" \
        ip saddr 10.77.30.20 \
        ip daddr 10.77.10.10 \
        tcp dport { 1514,1515 } \
        accept

        iifname "<GW_ATTACK_IF>" \
        oifname "<GW_MGMT_IF>" \
        drop
    }

    chain output {
        type filter hook output priority 0;
        policy accept;
    }
}
```

Validation

```
sudo nft -c -f /etc/nftables.conf
sudo nft -f /etc/nftables.conf
sudo nft list ruleset
sudo systemctl restart nftables
```

TC-NET-002

[soc-attacker]

```
ping -c 4 10.77.30.20
```

Expected:

```
SUCCESS
```

TC-NET-003

Attacker → MGMT

```
ping -c 3 10.77.10.10
```

Expected:

```
FAIL
```

Evidence

```
EV-NET-001
EV-FW-001
```

Rollback

```
sudo cp /etc/nftables.conf /etc/nftables.conf.failed
```

임시 Test 시에만:

```
[DESTRUCTIVE - FIREWALL POLICY REMOVAL]
sudo nft flush ruleset
```

실제 운영은 즉시 안전한 Baseline Rule을 재적용한다.

Completion Gate

GATE-FW-01

```
Attack → Victim   ALLOW
Attack → MGMT     DENY
```

12. Phase 7 — Hyper-V Port Mirroring

Microsoft 공식 cmdlet은 `-PortMirroring Source`와 `Destination`을 지원하며 Source와 Destination Adapter가 동일 Virtual Switch에 연결되어 있어야 한다.

Source

```
soc-victim
nic-victim
soc-vsw-victim
```

Destination

```
soc-sensor
nic-monitor
soc-vsw-victim
```

[PowerShell - Administrator]

```
Set-VMNetworkAdapter `
 -VMName "soc-victim" `
 -Name "nic-victim" `
 -PortMirroring Source

Set-VMNetworkAdapter `
 -VMName "soc-sensor" `
 -Name "nic-monitor" `
 -PortMirroring Destination
```

검증:

```
Get-VMNetworkAdapter `
 -VMName "soc-victim","soc-sensor" |
Select VMName,Name,SwitchName,PortMirroring,MacAddress
```

Expected:

```
soc-victim nic-victim   Source
soc-sensor nic-monitor  Destination
```

Rollback

```
Set-VMNetworkAdapter `
 -VMName "soc-victim" `
 -Name "nic-victim" `
 -PortMirroring None

Set-VMNetworkAdapter `
 -VMName "soc-sensor" `
 -Name "nic-monitor" `
 -PortMirroring None
```

GATE-MIRROR-CONFIG-01

PortMirroring Mode가 정확해야 한다.

---

13. Phase 8 — Packet Visibility Validation

CRITICAL PHASE

Suricata보다 먼저 검증한다.

Sensor

[Bash - soc-sensor]

```
sudo apt install -y tcpdump
```

```
sudo tcpdump \
 -ni <SENSOR_MONITOR_IF> \
 host 10.77.20.20 and host 10.77.30.20
```

Attacker

[Bash - soc-attacker]

```
ping -c 4 10.77.30.20
```

Victim HTTP가 준비되었다면:

```
curl <http://10.77.30.20/>
```

Expected

Sensor Console에 다음이 나타나야 한다.

```
10.77.20.20
↔
10.77.30.20
```

Sensor `nic-monitor` 자체에는 IP가 없어야 한다.

GATE-NET-01 — Packet Visibility

```
Traffic generation          PASS
Victim receives             PASS
Mirror Source               PASS
Mirror Destination          PASS
Sensor tcpdump              PASS
```

이 Gate가 FAIL이면 **Phase 9 진입 금지**.

Failure Tree

```
Attacker Traffic?
 ↓
Gateway Route?
 ↓
Gateway Firewall?
 ↓
Victim Receive?
 ↓
Mirror Source?
 ↓
Mirror Destination?
 ↓
Same vSwitch?
 ↓
Linux Interface Mapping?
```

```
 ↓
tcpdump?
```

Evidence

```
EV-MIRROR-001
```

반드시 Screenshot + tcpdump 출력 저장.

14. Phase 9 — Suricata Deployment

Objective

Sensor에 Baseline인 Suricata 8.0.6을 설치한다.

Suricata 공식 Ubuntu 설치 방법은 OISF `suricata-stable` PPA를 사용한다.

Temporary Maintenance Access

Sensor에 Internet Package Access가 필요할 경우에만 임시 NIC를 사용한다.

[PowerShell - Administrator]

먼저 Default Switch 존재 확인:

```
Get-VMSwitch -Name "Default Switch"
```

존재하는 경우:

```
Add-VMNetworkAdapter `
 -VMName "soc-sensor" `
 -Name "nic-maint" `
 -SwitchName "Default Switch"
```

설치가 완료되면 반드시 제거한다.

Install

[Bash - soc-sensor]

```
sudo apt update
sudo apt install -y software-properties-common jq
sudo add-apt-repository -y ppa:oisf/suricata-stable
sudo apt update
```

Package Candidate 확인:

```
apt-cache policy suricata
```

8.0.6 Candidate 확인 후:

```
sudo apt install -y suricata
```

검증:

```
suricata --build-info
suricata --version
```

Expected:

```
8.0.6
```

현재 다른 버전만 제공된다면:

```
VERSION-DRIFT-001
```

을 생성하고 중단한다.

Version Hold

Baseline 재현을 위해:

```
sudo apt-mark hold suricata
```

Maintenance NIC 제거

[PowerShell - Administrator]

```
Remove-VMNetworkAdapter `
 -VMName "soc-sensor" `
 -Name "nic-maint"
```

Evidence

```
EV-SURI-INSTALL-001
```

GATE-SURI-INSTALL-01

```
Suricata installed     PASS
Version 8.0.6          PASS
Maintenance NIC gone   PASS
```

---

15. Phase 10 — Suricata Baseline Configuration

Backup

[soc-sensor]

```
sudo cp \
 /etc/suricata/suricata.yaml \
 /etc/suricata/suricata.yaml.pre-soc
```

Network Variable

```
HOME_NET: "[10.77.30.0/24]"
EXTERNAL_NET: "!$HOME_NET"
```

AF_PACKET

```
af-packet:
  - interface: <SENSOR_MONITOR_IF>
    cluster-id: 99
    cluster-type: cluster_flow
    defrag: yes
```

Suricata 8 계열은 AF_PACKET을 공식 지원하며 non-inline 환경에서 관련 기본 설정이 개선되어 있다.

Rules

공식 managed rule path:

```
default-rule-path: /var/lib/suricata/rules
```

```
rule-files:
  - suricata.rules
  - /etc/suricata/rules/soc-network.rules
  - /etc/suricata/rules/soc-web.rules
```

```
  - /etc/suricata/rules/soc-auth.rules
  - /etc/suricata/rules/soc-lab.rules
```

Suricata 공식 문서는 `suricata-update` 기본 Rule 위치를 `/var/lib/suricata/rules/suricata.rules`로 정의하며 Custom Rule은 절대경로로 추가할 수 있다.

Create Rule Files

```
sudo install -d -m 0755 /etc/suricata/rules

sudo touch /etc/suricata/rules/soc-network.rules
sudo touch /etc/suricata/rules/soc-web.rules
sudo touch /etc/suricata/rules/soc-auth.rules
sudo touch /etc/suricata/rules/soc-lab.rules

sudo chmod 0644 /etc/suricata/rules/*.rules
```

Rule Update

```
sudo suricata-update
```

확인:

```
ls -lh /var/lib/suricata/rules/suricata.rules
```

EVE

활성화 대상:

```
alert
flow
http
dns
tls
stats
```

Suricata EVE는 `timestamp`, `event_type`, source/destination, protocol과 alert signature 등의 JSON 필드를 제공한다.

Configuration Test

```
sudo suricata -T \
 -c /etc/suricata/suricata.yaml
```

GATE-SURI-01

```
Configuration parse      PASS
Managed rules            PASS
Custom rule files        PASS
AF_PACKET interface      PASS
No duplicate SID         PASS
```

---

16. Phase 11 — Suricata Detection Validation

Deterministic Test Rule

`/etc/suricata/rules/soc-lab.rules`

```
alert icmp $EXTERNAL_NET any -> $HOME_NET any (msg:"SOC LAB ICMP TEST"; itype:8; sid:9030001; rev:1;)
```

Test

```
sudo suricata -T -c /etc/suricata/suricata.yaml
sudo systemctl restart suricata
sudo systemctl status suricata --no-pager
```

Attacker

```
ping -c 4 10.77.30.20
```

Sensor

```
sudo tail -f /var/log/suricata/fast.log
```

다른 Console:

```
sudo tail -f /var/log/suricata/eve.json |
jq 'select(.event_type=="alert")'
```

Expected EVE

```
src_ip         10.77.20.20
dest_ip        10.77.30.20
event_type     alert
```

```
signature_id   9030001
signature      SOC LAB ICMP TEST
```

GATE-DETECT-01

```
Traffic      PASS
Rule Match   PASS
fast.log     PASS
eve.json     PASS
```

Evidence

```
EV-SURI-003
```

포함:

```
Rule
Packet
EVE event
Screenshot
```

17. Phase 12 — PCAP Evidence Pipeline

Directory

```
sudo mkdir -p /var/lib/soc-lab/pcaps/{raw,scenarios,validated,archive}
sudo chmod 0755 /var/lib/soc-lab/pcaps
```

Capture

```
sudo tcpdump \
 -ni <SENSOR_MONITOR_IF> \
 -w /var/lib/soc-lab/pcaps/scenarios/PCAP-20260824-ATK-001.pcap \
 host 10.77.20.20 and host 10.77.30.20
```

Generate Traffic:

```
ping -c 4 10.77.30.20
```

Stop tcpdump.

Hash

```
sha256sum \
 /var/lib/soc-lab/pcaps/scenarios/PCAP-20260824-ATK-001.pcap
```

Metadata에 Hash를 저장한다.

PCAP Check

```
tcpdump -nn \
 -r /var/lib/soc-lab/pcaps/scenarios/PCAP-20260824-ATK-001.pcap
```

GATE-PCAP-01

```
PCAP exists
Packets readable
Source/destination correct
SHA256 recorded
```

---

18. Phase 13 — Snort Deployment

Objective

Snort 3.12.2.0을 **Secondary Offline Validation Engine**으로 구축한다.

Snort 공식 다운로드 페이지는 현재 Snort 3.12.2.0과 libDAQ 3.0.27을 제공한다.

Source Preparation

공식 Snort Download에서 다음 파일을 받은 후 Sensor의 `/tmp/soc-build/`에 둔다.

```
libdaq-3.0.27.tar.gz
snort3-3.12.2.0.tar.gz
```

Dependencies

[soc-sensor]

```
sudo apt update

sudo apt install -y \
 build-essential \
```

```
  cmake \
  autoconf \
  libtool \
  pkg-config \
  libpcap-dev \
  libpcre3-dev \
  libnet1-dev \
  zlib1g-dev \
  liblzma-dev \
  libssl-dev \
  libhwloc-dev \
  libluajit-5.1-dev \
  libunwind-dev \
  libmnl-dev \
  liblz4-dev \
  libnuma-dev
```

Build libDAQ

```
cd /tmp/soc-build
tar xf libdaq-3.0.27.tar.gz
cd libdaq-3.0.27

./bootstrap
./configure
make -j"$(nproc)"
sudo make install
sudo ldconfig
```

Build Snort

```
cd /tmp/soc-build
tar xf snort3-3.12.2.0.tar.gz
cd snort3-3.12.2.0

./configure_cmake.sh --prefix=/usr/local
cd build
make -j"$(nproc)"
sudo make install
sudo ldconfig
```

Verification

```
/usr/local/bin/snort -V
```

Expected:

```
3.12.2.0
```

GATE-SNORT-INSTALL-01

```
libDAQ 3.0.27   PASS
Snort 3.12.2.0 PASS
```

---

19. Phase 14 — Snort Offline Validation

Snort 공식 문서는 `-r` PCAP readback과 `alert_json` Logger를 지원한다.

Config Test

```
sudo /usr/local/bin/snort \
 -c /usr/local/etc/snort/snort.lua \
 -T
```

Local Rule

```
sudo mkdir -p /usr/local/etc/snort/rules
```

```
echo 'alert icmp any any -> 10.77.30.0/24 any (msg:"SOC LAB SNORT ICMP
TEST"; sid:9100001; rev:1;)' |
sudo tee /usr/local/etc/snort/rules/soc-local.rules
```

Offline Analysis

```
mkdir -p /var/log/snort

sudo /usr/local/bin/snort \
 -q \
 -c /usr/local/etc/snort/snort.lua \
 -R /usr/local/etc/snort/rules/soc-local.rules \
 -r /var/lib/soc-lab/pcaps/scenarios/PCAP-20260824-ATK-001.pcap \
```

```
  -A alert_json \
  -l /var/log/snort \
  --lua "alert_json = {file = true, fields = 'timestamp pkt_num proto sr
c_ap dst_ap rule action msg class'}"
```

Snort의 `alert_json`은 JSON 출력과 별도 Log Directory 저장을 지원한다.

Compare

```
Suricata SID   9030001
Snort SID      9100001
PCAP           동일
Traffic        동일
Result         둘 다 Detection
```

GATE-SNORT-01

동일 PCAP에 대해 Suricata와 Snort Detection Result를 비교할 수 있어야 한다.

Evidence:

```
EV-SNORT-001
```

---

20. Phase 15 — Wazuh Host Readiness

Wazuh 공식 single-node Docker 요구사항은 최소 **4 cores / 8GB RAM / 50GB disk**이며 WSL2 환경에서도 `vm.max_map_count=262144`를 요구한다.

[PowerShell]

```
docker version
docker compose version
wsl --version
```

[WSL2]

```
sysctl vm.max_map_count
```

Expected:

```
>= 262144
```

필요 시:

```
sudo sysctl -w vm.max_map_count=262144
```

Persistence:

```
echo 'vm.max_map_count=262144' |
sudo tee /etc/sysctl.d/99-wazuh.conf
```

Compose Version

LLD의 Port Binding Override에 `!override`를 사용할 경우 Docker Compose **2.24.4 이상**이어야 한다. Docker 공식 Compose merge 규칙에서 `!override`가 2.24.4+ 기능으로 명시되어 있다.

```
docker compose version
```

GATE-WAZUH-HOST-01

```
Docker operational
Compose compatible
WSL compatible
vm.max_map_count >= 262144
RAM available
Disk available
```

---

21. Phase 16 — Wazuh Docker Deployment

Clone Pinned Release

GitHub CLI 사용:

```
Set-Location C:\SOC-Lab\artifacts

gh repo clone wazuh/wazuh-docker -- --branch v4.14.7
```

공식 Wazuh 문서 역시 `v4.14.7` branch를 사용한 single-node Clone을 지시한다.

WSL

```
cd /mnt/c/SOC-Lab/artifacts/wazuh-docker/single-node
```

Certificate Generation

```
docker compose \
 -f generate-indexer-certs.yml \
 run --rm generator
```

21.1 Compose Override

파일:

```
docker-compose.override.yml
```

```
services:

  wazuh.manager:
    container_name: soc-wazuh-manager
    ports: !override
      - "10.77.10.10:1514:1514"
      - "10.77.10.10:1515:1515"
      - "127.0.0.1:55000:55000"

  wazuh.indexer:
    container_name: soc-wazuh-indexer
    ports: !override
      - "127.0.0.1:9200:9200"

  wazuh.dashboard:
    container_name: soc-wazuh-dashboard
    ports: !override
      - "10.77.10.10:443:5601"

networks:
  default:
    name: soc-wazuh-net
```

Compose의 `!override`는 기존 Port Sequence를 완전히 대체하기 위한 공식 기능이다.

Effective Config 확인

```
docker compose config > /tmp/soc-wazuh-effective.yml
```

확인:

```
grep -nE '1514|1515|55000|9200|443|soc-wazuh' \
 /tmp/soc-wazuh-effective.yml
```

Deploy

```
docker compose up -d
```

공식 Wazuh Docker single-node 배포도 `docker compose up -d`를 사용하며 manager/indexer/dashboard가 각각 별도 Container로 배포된다.

Check

```
docker compose ps
docker ps
```

Expected:

```
soc-wazuh-manager
soc-wazuh-indexer
soc-wazuh-dashboard
```

21.2 Security Hardening — Default Password

공식 Wazuh Docker는 bootstrap 계정에 기본 Credential을 사용하므로 **GATE-WAZUH-01 전 반드시 변경**한다. Wazuh도 공식적으로 default password 변경을 권장한다.

Password Hash Generator:

```
docker run --rm -it \
 wazuh/wazuh-indexer:4.14.7 \
 bash \
 /usr/share/wazuh-indexer/plugins/opensearch-security/tools/hash.sh
```

새 Password를 다음 Runtime 파일과 필요한 Wazuh 설정에 반영한다.

```
docker-compose.yml / override
config/wazuh_indexer/internal_users.yml
config/wazuh_dashboard/wazuh.yml
```

Password 자체는 Repository에 Commit하지 않는다.

재기동:

```
docker compose down
docker compose up -d
```

Dashboard

```
<https://10.77.10.10/>
```

GATE-WAZUH-01

```
Indexer responsive       PASS
Manager running          PASS
Dashboard accessible     PASS
Login successful         PASS
Default passwords gone   PASS
```

22. Phase 17 — Sensor Wazuh Agent

Objective

Sensor의 Suricata EVE를 Wazuh로 전송할 Agent를 등록한다.

Wazuh의 기본 Agent 통신은 1514/TCP, 자동 Enrollment는 1515/TCP이다.

Connectivity First

[soc-sensor]

```
nc -zv 10.77.10.10 1514
nc -zv 10.77.10.10 1515
```

Installation

Wazuh Dashboard:

```
Agents management
→ Summary
→ Deploy new agent
```

선택:

```
OS: Ubuntu/Debian
Manager: 10.77.10.10
Agent Name: soc-sensor
```

Dashboard가 생성한 **현재 4.14.7 공식 설치 명령**을 Sensor에서 실행한다.

이 방식은 Repository bootstrap command drift를 방지하기 위한 것이다.

Verify

```
sudo systemctl status wazuh-agent --no-pager
```

GATE-AGENT-SENSOR-01

Dashboard에서:

```
soc-sensor
Status = Active
```

---

23. Phase 18 — Victim Wazuh Agent

Firewall Precheck

Gateway에서 Victim→Host의 다음 포트만 허용되어 있어야 한다.

```
1514/TCP
1515/TCP
```

Connectivity

[soc-victim]

```
nc -zv 10.77.10.10 1514
nc -zv 10.77.10.10 1515
```

Installation

Dashboard:

```
Agent Name:
soc-victim
```

```
Manager:
10.77.10.10
```

Vendor-generated 4.14.7 install command 사용.

Gate

GATE-AGENT-VICTIM-01

```
soc-victim = Active
```

24. Phase 19 — Suricata → Wazuh Integration

Wazuh 공식 Suricata Integration은 Wazuh Agent가 `/var/log/suricata/eve.json`을 `json` 형식으로 Localfile 수집하는 구조를 사용한다.

Backup

[soc-sensor]

```
sudo cp \
 /var/ossec/etc/ossec.conf \
 /var/ossec/etc/ossec.conf.pre-suricata
```

Add Localfile Block

다음 Python을 사용해 기존 설정을 보존하면서 Block을 추가한다.

```
sudo python3 - <<'PY'
from pathlib import Path

p = Path("/var/ossec/etc/ossec.conf")
s = p.read_text()

marker = "/var/log/suricata/eve.json"

block = """
  <localfile>
    <log_format>json</log_format>
    <location>/var/log/suricata/eve.json</location>
  </localfile>
"""
```

```
if marker not in s:
    pos = s.rfind("</ossec_config>")
    if pos == -1:
        raise SystemExit("ERROR: closing ossec_config tag not found")
    s = s[:pos] + block + "\n" + s[pos:]
    p.write_text(s)
else:
    print("Suricata localfile already configured")
PY
```

Restart Agent

```
sudo systemctl restart wazuh-agent
sudo systemctl status wazuh-agent --no-pager
```

공식 PoC 역시 Agent 설정에 EVE Localfile을 추가한 뒤 Wazuh Agent를 재시작한다.

Generate Alert

Attacker

```
ping -c 4 10.77.30.20
```

Sensor

```
jq 'select(
  .event_type=="alert" and
  .alert.signature_id==9030001
)' /var/log/suricata/eve.json
```

Dashboard

검색:

```
agent.name:soc-sensor
```

그 후 Source IP:

```
10.77.20.20
```

확인.

GATE-SIEM-01

```
Attack
  PASS
Suricata
  PASS
eve.json
  PASS
Agent
  PASS
Wazuh
  PASS
Dashboard Alert
  PASS
```

25. Phase 20 — Dashboard Validation

Objective

SOC Analyst가 필요한 핵심 필드를 Dashboard에서 조회할 수 있음을 검증한다.

Search Fields

```
Timestamp
Agent
Source IP
Destination IP
Rule
Signature
Protocol
```

Validation Scenario

```
ATK-001
Source: 10.77.20.20
Destination: 10.77.30.20
SID: 9030001
```

GATE-DASHBOARD-01

동일 Event를

```
eve.json
↔
Wazuh Dashboard
```

에서 상호 확인할 수 있어야 한다.

---

26. Phase 21 — Attack Scenario Execution

모든 시나리오는 **10.77.0.0/16 Lab 안에서만 수행**한다.

외부 Public IP 대상 명령 실행 금지.

ATK-001 — ICMP

Attacker

```
ping -c 4 10.77.30.20
```

Expected:

```
SID 9030001
```

Purpose:

```
기본 Detection Pipeline 검증
```

ATT&CK:

```
직접 ATT&CK Attack Technique으로 분류하지 않음
```

단순 Ping 자체는 악성행위로 단정하지 않는다.

ATK-003 — Port Scan

MITRE ATT&CK T1046은 Port/Service Scan과 같은 Network Service Discovery를 포함한다.

Attacker

```
nmap -sT -Pn -p 22,80 10.77.30.20
```

ATT&CK Candidate

```
T1046
Network Service Discovery
Tactic: Discovery
```

Deterministic Lab Rule

`/etc/suricata/rules/soc-network.rules`

```
alert tcp 10.77.20.20 any -> 10.77.30.20 any (msg:"SOC LAB TCP SYN ACTIVITY"; flags:S; threshold:type bo
th, track by_src, count 5, seconds 10; sid:9000001; rev:1;)
```

Config 검증 후 Restart:

```
sudo suricata -T -c /etc/suricata/suricata.yaml
sudo systemctl restart suricata
```

ATK-005 — Authentication Failure

Victim SSH가 활성화된 경우 실행한다.

Attacker

```
ssh invalid-soc-user@10.77.30.20
```

잘못된 Credential을 수회 입력하여 실패 이벤트를 생성한다.

MITRE ATT&CK에서 반복적 Password Guessing은 `T1110.001 Password Guessing`에 해당할 수 있다.

Mapping

```
T1110.001
Password Guessing
Credential Access
```

단 실제 Scenario가 반복 추측 행동을 포함할 때만 해당 Mapping을 사용한다.

ATK-007 — Synthetic Suspicious HTTP

실제 공격 Payload 대신 **harmless marker**를 사용한다.

Victim HTTP:

```
<http://10.77.30.20/>
```

Initial Rule

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"SOC LAB HTTP GET BASELINE"; flow:to_server,established; http.method; content:"GET"; sid:9010001; rev:1;)
```

Attacker:

```
curl <http://10.77.30.20/>
```

이 Rule은 정상 GET까지 Alert하므로 이후 FP Tuning Scenario에 사용한다.

ATK-012 — Malware PCAP

실제 Malware를 실행하지 않는다.

```
Approved / Sanitized PCAP
        ↓
Suricata Offline
        ↓
Snort Offline
        ↓
Wireshark
```

PCAP:

```
/var/lib/soc-lab/pcaps/validated/<approved-malware-traffic>.pcap
```

ATT&CK Mapping은 PCAP의 실제 행동 Evidence를 분석한 뒤 수행한다.

GATE-ATTACK-01

최소 다음 5개 Scenario에 대해 실행 또는 Offline Validation 자료가 존재해야 한다.

```
ATK-001
ATK-003
ATK-005
ATK-007
ATK-012
```

---

27. Phase 22 — SOC Investigation

각 Alert마다 다음 절차를 수행한다.

```
1. Timestamp
2. Sensor
3. Source IP
4. Destination IP
5. Protocol
```

```
6. SID
7. Signature
8. Raw EVE
9. Related PCAP
10. Packet Stream
11. Related Wazuh Events
12. ATT&CK
13. Verdict
14. Response
```

Incident Template

```
# INC-YYYYMMDD-NNN

## Detection

- Alert:
- SID:
- Timestamp:
- Sensor:

## Network

- Source:
- Destination:
- Protocol:
- Ports:

## Evidence

- EVE:
- PCAP:
- SHA256:
- Rule:

## Analysis

...

## MITRE ATT&CK

...

## Verdict

TRUE_POSITIVE / FALSE_POSITIVE / BENIGN / UNRESOLVED
```

```
## Response

...

## Detection Improvement

...
```

GATE-ANALYSIS-01

최소 하나의 Alert에 대해

```
Alert
→ EVE
→ PCAP
→ Rule
→ Verdict
```

연결이 완성되어야 한다.

Evidence:

```
EV-ANALYSIS-001
```

---

28. Phase 23 — MITRE ATT&CK Mapping

ATT&CK ID는 추측하여 부여하지 않는다.

현재 Scenario에서 명확하게 사용할 수 있는 Baseline:

Scenario	ATT&CK
ICMP Connectivity Test	없음
Port Scan	**T1046**
Password Guessing	**T1110.001**
Synthetic HTTP Marker	없음
Malware PCAP	분석 후 결정

Port Scan/Service Scan은 T1046, Password Guessing은 T1110.001 공식 정의와 직접 대응된다.

GATE-ATTACK-MAP-01

ATT&CK Mapping마다 다음 Evidence가 있어야 한다.

```
Observed Behavior
     ↓
```

```
ATT&CK Definition
     ↓
Technique ID
     ↓
Why it applies
```

29. Phase 24 — False Positive Analysis

Objective

정상 Traffic이 너무 광범위한 Rule에 의해 Alert되는 사례를 인위적으로 만든다.

Baseline Rule

```
SID:
9010001

Rule:
Any HTTP GET → Alert
```

Attacker

```
curl <http://10.77.30.20/>
```

이는 정상 단순 HTTP 조회지만 Alert된다.

Verdict

```
FALSE_POSITIVE
```

Root Cause

```
Detection Condition too broad
HTTP method GET only
No suspicious URI/content context
```

GATE-FP-01

False Positive의

```
Raw EVE
PCAP
```

```
Rule
Reason
```

이 모두 기록되어야 한다.

---

30. Phase 25 — Detection Rule Tuning

기존 Rule:

```
SID 9010001 rev:1
Any HTTP GET
```

을 수정한다.

Tuned Rule

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"SOC LAB SUSPICIOUS HTTP MARKER"; flow:to_server,esta
blished; http.uri; content:"/soc-lab-suspicious"; sid:9010001; rev:2;)
```

Configuration Test

```
sudo suricata -T -c /etc/suricata/suricata.yaml
```

Git

Repository의 동일 Rule도 변경한다.

```
git add suricata/rules
git commit -m "feat(detection): tune HTTP lab rule to reduce false posi
tives"
```

GATE-TUNE-CONFIG-01

```
Revision increased
Config valid
Git commit exists
```

---

31. Phase 26 — Re-test

Normal Traffic

Attacker

```
curl <http://10.77.30.20/>
```

Expected:

```
SID 9010001
NO ALERT
```

Suspicious Marker

```
curl <http://10.77.30.20/soc-lab-suspicious>
```

Expected:

```
SID 9010001 rev 2
ALERT
```

Target Web Server가 404를 반환하더라도 HTTP Request 자체는 Suricata가 관찰할 수 있다.

GATE-TUNE-01

반드시 두 조건을 동시에 만족한다.

```
Normal GET
→ Alert 제거

Marker GET
→ Alert 유지
```

Evidence:

```
EV-TUNE-001
```

이 단계가 Portfolio의 핵심 Detection Engineering 증적이다.

---

32. Phase 27 — End-to-End Validation

다음 Scenario를 하나 선정한다.

권장:

```
ATK-003 Port Scan
```

Execution

Attacker

```
nmap -sT -Pn -p 22,80 10.77.30.20
```

Validation Chain

```
1. Victim Packet
2. Sensor tcpdump
3. Suricata Alert
4. eve.json
5. Wazuh Event
6. Dashboard
7. PCAP
8. Rule
9. T1046
10. Verdict
11. Incident Report
12. Evidence
```

GATE-E2E-01

모든 계층이 한 Incident ID에 연결되어야 한다.

예:

```
INC-20260824-001
├─ ATK-003
├─ SID-9000001
├─ PCAP-20260824-ATK-003
├─ T1046
├─ EV-SURI-xxx
├─ EV-WAZUH-xxx
└─ EV-ANALYSIS-xxx
```

33. Phase 28 — Evidence Completion

Evidence Directory

```
evidence/
├── EV-HOST-001/
├── EV-NET-001/
├── EV-MIRROR-001/
```

```
├── EV-SURI-001/
├── EV-PCAP-001/
├── EV-SNORT-001/
├── EV-WAZUH-001/
├── EV-ANALYSIS-001/
└── EV-TUNE-001/
```

metadata.md

```
# Evidence Metadata

Evidence ID:
Requirement:
Design:
Implementation Phase:
Test:
Scenario:
Timestamp:
Component:

## Expected

...

## Actual

...

## Result

PASS / FAIL

## Related

PCAP:
Rule:
Incident:
ATT&CK:
Git Commit:
```

PCAP Integrity

```
sha256sum <pcap>
```

Hash를 Metadata에 기록한다.

GATE-EVIDENCE-01

핵심 P0/P1 Test에 Evidence 누락이 없어야 한다.

---

34. Phase 29 — GitHub Portfolio Documentation

README는 설치 설명이 아니라 **SOC 분석 역량** 중심으로 구성한다.

```
1. Problem
2. Objective
3. Architecture
4. Network Topology
5. Packet Visibility
6. Suricata
7. Snort Comparison
8. Wazuh SIEM
9. Attack Scenarios
10. Detection Rules
11. SOC Investigation
12. MITRE ATT&CK
13. False Positive
14. Rule Tuning
15. Before / After
16. Evidence
17. Troubleshooting
18. Lessons Learned
```

핵심 Portfolio Figure

```
ATTACKER
   ↓
VICTIM
   ↓ Mirror
SURICATA
   ↓ EVE
WAZUH
   ↓
SOC ANALYST
   ↓
PCAP + RULE + ATT&CK
   ↓
VERDICT
   ↓
TUNING
```

Secret Check

Git Commit 전:

```
git status
git diff --cached
```

다음 문자열이 존재하지 않는지 확인:

```
password
SecretPassword
API key
private key
token
credential
```

실제 Secret이 포함된 경우 Commit 금지.

GATE-PORTFOLIO-01

제3자가 README만 읽고 다음을 설명할 수 있어야 한다.

```
어떤 Traffic을 만들었는가?
어디서 탐지했는가?
어떤 Rule이 탐지했는가?
어떤 Log가 생성됐는가?
왜 실제 공격이라고 판단했는가?
ATT&CK은 무엇인가?
오탐을 어떻게 줄였는가?
```

---

35. Phase 30 — Final Release Gate

GATE-RELEASE-01

다음 전체 Checklist를 검증한다.

```
[ ] GATE-HOST-01 PASS
[ ] GATE-REPO-01 PASS
[ ] GATE-NET-INFRA-01 PASS
[ ] GATE-VM-01 PASS
[ ] GATE-IP-01 PASS
[ ] GATE-ROUTE-01 PASS
[ ] GATE-FW-01 PASS
[ ] GATE-MIRROR-CONFIG-01 PASS
[ ] GATE-NET-01 PASS
[ ] GATE-SURI-INSTALL-01 PASS
[ ] GATE-SURI-01 PASS
[ ] GATE-DETECT-01 PASS
[ ] GATE-PCAP-01 PASS
[ ] GATE-SNORT-INSTALL-01 PASS
[ ] GATE-SNORT-01 PASS
[ ] GATE-WAZUH-HOST-01 PASS
[ ] GATE-WAZUH-01 PASS
[ ] GATE-AGENT-SENSOR-01 PASS
[ ] GATE-AGENT-VICTIM-01 PASS
[ ] GATE-SIEM-01 PASS
[ ] GATE-DASHBOARD-01 PASS
[ ] GATE-ATTACK-01 PASS
[ ] GATE-ANALYSIS-01 PASS
[ ] GATE-ATTACK-MAP-01 PASS
```

```
[ ] GATE-FP-01 PASS
[ ] GATE-TUNE-CONFIG-01 PASS
[ ] GATE-TUNE-01 PASS
[ ] GATE-E2E-01 PASS
[ ] GATE-EVIDENCE-01 PASS
[ ] GATE-PORTFOLIO-01 PASS
```

36. Runtime Verification Matrix

Gate	확인	Command
RG-001	Windows Build	`Get-ComputerInfo`
RG-002	WSL Version	`wsl --version`
RG-003	Gateway Interface	`ip link`
RG-004	Sensor Interface	`ip link`
RG-005	Hyper-V MAC	`Get-VMNetworkAdapter`
RG-006	Docker Bind	`docker compose config`

RG-003~005 결과는 Repository의

```
docs/04-deployment/runtime-baseline.md
```

에 확정값으로 저장한다.

37. Gate Matrix

Gate	핵심 검증	Evidence
HOST	Host 준비	EV-HOST-001
NET	통신	EV-NET-001
FW	격리	EV-FW-001
MIRROR	Packet 복제	EV-MIRROR-001
SURI	IDS 정상	EV-SURI-001
DETECT	Alert	EV-SURI-003
PCAP	Packet Evidence	EV-PCAP-001
SNORT	Secondary Validation	EV-SNORT-001
WAZUH	SIEM 정상	EV-WAZUH-001
SIEM	EVE Integration	EV-WAZUH-002
ANALYSIS	SOC 조사	EV-ANALYSIS-001
TUNE	오탐 개선	EV-TUNE-001
E2E	전체 Pipeline	EV-E2E-001
PORTFOLIO	문서 완성	EV-PORTFOLIO-001

38. Dependency Matrix

Phase	Dependency
1	Phase 0
2	Phase 0
3	Phase 2
4	Phase 3
5	Phase 4
6	Phase 5
7	Phase 3
8	Phase 5,6,7
9	**Phase 8 PASS**
10	Phase 9
11	Phase 10
12	Phase 8
13	Phase 12
14	Phase 13
15	Phase 0
16	Phase 15
17	Phase 16
18	Phase 16
19	Phase 11,17
20	Phase 19
21	Phase 20
22	Phase 21
23	Phase 22
24	Phase 21
25	Phase 24
26	Phase 25
27	Phase 26
28	모든 기술 Gate
29	Phase 28
30	Phase 29

39. Rollback Matrix

Phase	Rollback 핵심
vSwitch	`Remove-VMSwitch`

Phase	Rollback 핵심
VM	VM Snapshot 또는 VM 삭제
IP	Netplan Backup 복원
Routing	sysctl 설정 제거
Firewall	이전 nftables.conf 복원
Mirror	PortMirroring None
Suricata	`suricata.yaml.pre-soc` 복원
Rule	Git Checkout
Wazuh	`docker compose down`
Wazuh Config	Vendor Baseline 복원
Agent	ossec.conf Backup 복원
Tuning	이전 Rule Revision
Repository	Git Revert

40. Troubleshooting Record

모든 장애는 다음 Template을 사용한다.

```
# TRB-NNN

Phase:
Date:
Component:

## Symptom

...

## Expected

...

## Actual

...

## Layer

Host / Network / Mirror / IDS / Log / Agent / Manager / Indexer / Dashb
oard

## Commands
```

```
...

## Root Cause

...

## Resolution

...

## Re-test

...

## Evidence

...
```

41. 가장 중요한 Troubleshooting Tree

Suricata Alert 없음

```
Traffic exists?
     ↓
Victim receives?
     ↓
Mirror configured?
     ↓
Sensor tcpdump?
     ↓
Suricata service?
     ↓
Correct monitor NIC?
     ↓
AF_PACKET?
     ↓
Rule loaded?
     ↓
Rule matches?
     ↓
EVE enabled?
```

Suricata에는 있으나 Wazuh에는 없음

```
eve.json?
   ↓
Wazuh Agent?
   ↓
localfile?
```

```
   ↓
1514?
   ↓
Agent Active?
   ↓
Manager?
   ↓
Indexer?
   ↓
Dashboard Query?
```

---

42. Change Management

LLD Baseline 수정이 필요하면 직접 수정하지 않는다.

```
DESIGN-ISSUE-001
```

Template:

```
Original:
Observed Problem:
Proposed:
Reason:
Security Impact:
Testing Impact:
Decision:
```

Architecture까지 영향을 미치면:

```
ADR-CANDIDATE-001
```

로 승격한다.

---

43. Task Status

허용값:

```
TODO
IN_PROGRESS
PASS
FAIL
BLOCKED
SKIPPED
```

P0 Task는 `SKIPPED` 허용하지 않는다.

---

44. Phase Result Template

실제 구축 시 모든 Phase 종료마다 다음 형식으로 기록한다.

```
# Phase Result

Phase:
Date:

Status:
PASS / FAIL / BLOCKED

## Completed

...

## Runtime Values

...

## Tests

...

## Evidence

...

## Issues

...

## Changes

...

## Git Commit

...

## Next Phase

...
```

45. Requirement → Design → Implementation → Test → Evidence

영역	Requirement	Design	Implementation	Test	Evidence
Traffic	FR-IDS-001	DES-NET-001	P4~8	TC-NET-002	EV-NET-001
Mirroring	FR-IDS-001	DES-MIRROR-001	P7~8	TC-MIRROR-001	EV-MIRROR-001
Suricata	FR-IDS-002	DES-SURI-001	P9~11	TC-SURI-003	EV-SURI-003
PCAP	FR-IDS-004	DES-PCAP-001	P12	TC-PCAP-001	EV-PCAP-001
Snort	FR-IDS-003	DES-SNORT-001	P13~14	TC-SNORT-001	EV-SNORT-001
SIEM	FR-SIEM-001	DES-WAZUH-001	P15~20	TC-WAZUH-002	EV-WAZUH-001
Analysis	FR-ANALYSIS	DES-ANALYSIS	P22	TC-ANALYSIS-001	EV-ANALYSIS-001
Tuning	FR-RULE	DES-TUNE	P24~26	TC-TUNE-001	EV-TUNE-001

46. Final Definition of Done

프로젝트는 다음 상태가 되어야 완성이다.

```
Packet Visibility
       PASS
        ↓
Suricata Detection
       PASS
        ↓
PCAP Evidence
       PASS
        ↓
Snort Validation
       PASS
        ↓
Wazuh Integration
       PASS
        ↓
SOC Investigation
       PASS
        ↓
MITRE ATT&CK
       PASS
        ↓
False Positive Analysis
       PASS
        ↓
Detection Tuning
       PASS
        ↓
Re-test
       PASS
        ↓
Evidence
       PASS
        ↓
GitHub Portfolio
       PASS
```

47. Implementation Completion State

모든 Gate가 통과되었을 때만:

```
IMPLEMENTATION COMPLETE
```

를 선언한다.

그 이전 상태는:

```
IMPLEMENTATION IN PROGRESS
```

또는

```
IMPLEMENTATION BLOCKED
```

로 표현한다.

---

48. 최초 실제 실행 지점

Implementation Plan 작성 이후 더 이상의 선행 설계문서는 작성하지 않는다.

첫 실제 작업은:

`Phase 0 — Host Readiness`

이다.

가장 먼저 실행할 Command Set은 다음이다.

```
Get-ComputerInfo |
Select-Object WindowsProductName,WindowsVersion,OsBuildNumber

Get-WindowsOptionalFeature `
 -Online `
 -FeatureName Microsoft-Hyper-V-All

Get-VMHost

wsl --version
wsl -l -v

docker version
docker compose version

git --version
```

```
Get-CimInstance Win32_ComputerSystem |
Select-Object TotalPhysicalMemory

Get-Volume |
Select-Object DriveLetter,
@{N='FreeGB';E={[math]::Round($_.SizeRemaining/1GB,2)}}
```

이 결과를 기준으로:

```
GATE-HOST-01
```

을 판정한다.

---

49. 최종 구축 흐름

```
PHASE 0
Host
   ↓
PHASE 1
Repository
   ↓
PHASE 2
vSwitch
   ↓
PHASE 3
VM
   ↓
PHASE 4
IP
   ↓
PHASE 5
Routing
   ↓
PHASE 6
Firewall
   ↓
PHASE 7
Mirror
   ↓
PHASE 8
tcpdump
   ↓
GATE-NET-01
   ↓
PHASE 9
Suricata
   ↓
PHASE 11
Detection
   ↓
PHASE 12
PCAP
   ↓
PHASE 14
Snort
   ↓
```

```
PHASE 16
Wazuh
   ↓
PHASE 19
EVE Integration
   ↓
PHASE 21
Attack
   ↓
PHASE 22
SOC Analysis
   ↓
PHASE 24
False Positive
   ↓
PHASE 25
Tuning
   ↓
PHASE 26
Re-test
   ↓
PHASE 27
E2E
   ↓
PHASE 28
Evidence
   ↓
PHASE 29
Portfolio
   ↓
PHASE 30
RELEASE
```

Implementation Plan v1.0 — FINAL BASELINE

> **본 Implementation Plan은 SOC Detection & Monitoring Lab을 Hyper-V 기반의 격리 Network에서 단계적으로 구축한다. 최초 핵심 Gate는 Suricata 설치가 아니라 Attacker → Gateway → Victim Traffic이 Hyper-V Port Mirroring을 통해 Sensor의 IP 없는 Monitoring NIC에 도달하는지 tcpdump로 검증하는 것이다. 해당 Gate 통과 후에만 Suricata 8.0.6을 AF_PACKET 기반 Primary IDS로 구축한다. Detection과 EVE가 로컬에서 정상 동작한 후에 Wazuh 4.14.7 Single-node를 배포하고 EVE를 Wazuh Agent로 수집한다. Snort 3.12.2.0은 동일 PCAP의 Secondary Validation Engine으로 사용한다. 이후 실제 Alert를 Raw EVE·PCAP·Rule·MITRE ATT&CK과 연결해 조사하고, False Positive Rule을 Tuning하여 정상 Traffic의 오탐은 제거하면서 목표 Detection은 유지되는지 Re-test한다. 최종적으로 Requirements → Design → Implementation → Test → Evidence 전 과정을 GitHub Portfolio에 증명한다.**

Implementation Plan Status:

`APPROVED — READY FOR PHASE 0`